

UrbanFlow Analytics

Architecture & Comprehensive Technical Report

From taxi trip records to fare estimates, arrival predictions, demand forecasts, and operational decisions.

Submission specification	Coverage
Preprocessing & feature engineering	One dedicated page (page 3)
Model development methodology	Pages 5–11
Evaluation metrics & results	Pages 6–10 and 12
Findings, business implications & conclusions	Pages 2, 12–13
Complete solution architecture diagram	Page 4
Total report length	14 pages, including cover and source register

Prepared 11 September 2026

Evidence basis: repository source code, saved notebook outputs, exported analytical reports, and a fresh aggregate-demand backtest. Previously saved model scores are identified as such; no model retraining was performed for this report.

02 / Executive summary

UrbanFlow combines predictive machine learning and descriptive analytics in one Streamlit application. Passengers can estimate a base fare and arrival time; dispatchers can inspect pickup demand and zone behavior; managers can explore revenue patterns and operational scenarios. A structured natural-language interface makes the underlying trip data accessible without requiring users to write SQL.

Component	Principal result	Evidence status
Data foundation	45,956,110 cleaned trips; 94.56% retained	Exported quality report; row count verified
Fare prediction	LightGBM saved test MAE \$4.5969	Notebook result; split audit required
Arrival estimation	Test MAE 6.8857 min; 81.48% within 10 min	Saved notebook evaluation
Fleet demand model	72-hour forecast for 10 zones; mean horizon WAPE 32.7%	Saved notebook evaluation
Dashboard demand baseline	Last-week aggregate WAPE 9.43%	Fresh fixed-origin backtest
Zone clustering	Five clusters; silhouette 0.2819	Exported diagnostic results

Main findings

Tree models improve fare validation error relative to the stored median and linear baselines. Arrival estimates are more reliable for shorter trips and substantially less reliable for journeys longer than 45 minutes. Demand varies by hour and zone, supporting differentiated staging decisions rather than one city-wide dispatch rule.

Business interpretation

The repository simulates \$25.97 million in annual driver benefit from tip-related assumptions and reduced empty mileage. This is a scenario output, not measured profit or a causal intervention result. Cash-tip under-recording, corridor asymmetry, and occupied-trip revenue metrics require careful interpretation before operational investment. [S8]

The strongest current deliverable is an integrated analytical prototype with usable dashboard modules and traceable model experiments. Reproducible temporal splits, complete model packaging, and stronger multi-period validation are the priorities before claiming production readiness.

03 / Data preprocessing & feature engineering

The intended observation window is 1 April 2025 to 31 March 2026. The exported quality summary records 48,601,782 raw rows, 45,956,110 cleaned rows, and 2,645,672 removals (5.44%). These figures supersede inconsistent README/dashboard totals. A fresh Parquet scan confirmed the cleaned count and found 13 timestamps outside the intended window. Temporal modeling must explicitly enforce the date boundary. [S1, S10]

Data issue	Affected raw rows	Recorded treatment
Negative base fare	2,400,031	Exclude from normal fare modeling
Zero distance with nonzero fare	1,471,745	Task-specific movement/speed filter
Zero rider count	231,578	Filter passenger-dependent analysis
Drop-off before pickup	1,942	Drop invalid temporal sequence
Speed above 80 mph	13,612	Filter movement/travel-time analysis

Anomaly categories can overlap and do not sum to total removals. The treatment matrix separates invalid targets from task-specific exclusions; it does not impute invented fares or rider counts. Cleaned/enriched Parquet supports column selection and aggregation without loading the entire dataset. Zone IDs join borough, zone-name, and service-zone metadata; absent metadata must remain visibly unknown. [S1]

Feature groups and availability

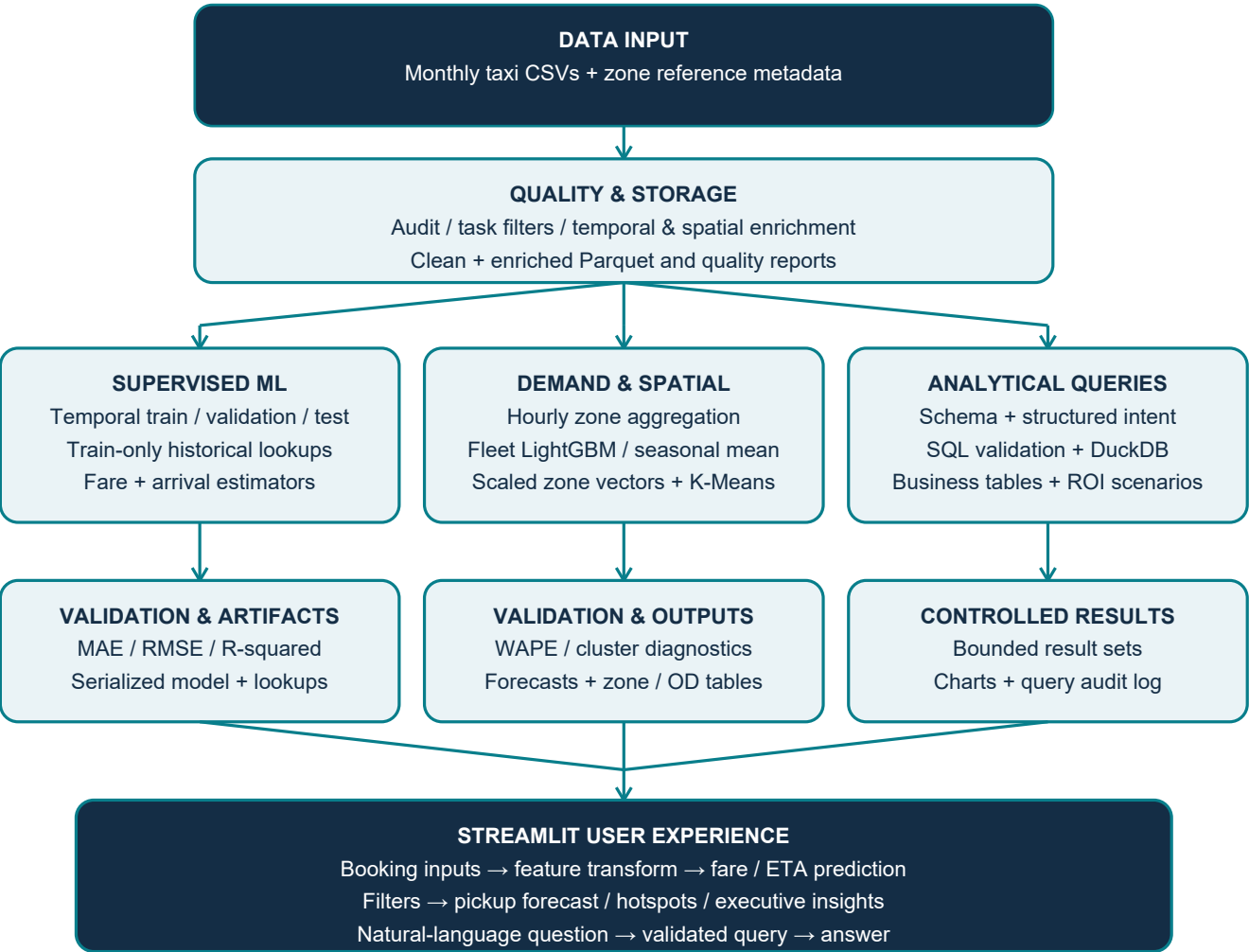
Fare: 23 inputs cover booking fields, origin/destination IDs, airport/cross-borough flags, hour/weekday/month, rush/weekend/night flags, cyclic time encodings, OD code, and training-only route medians. Distance is treated as a route-estimate proxy in the interface; historical recorded distance is a material pre-trip availability caveat. [S2]

Arrival: 26 inputs use booking, calendar, geographic categories, and training-only OD, OD-hour, and OD-weekday duration medians. Actual distance, speed, fare, settlement fields, and drop-off time are excluded from predictors; timestamp differences create the target. Valid modeling durations are positive and at most 180 minutes. [S3]

Demand/spatial: zero-filled hourly zone counts feed lagged and shifted rolling features; the dashboard averages matching weekday/hour slots. Zone clustering standardizes activity, daypart, route, and flow descriptors. Aggregate lookups must be fitted on training data only; transformations must be shared between training and inference. [S4–S6]

04 / Solution architecture

The system separates batch preparation and model fitting from interactive inference. Parquet is the shared analytical data layer; persisted estimators and lookup statistics carry training decisions into the application. The diagram includes both notebook experiments and implemented dashboard paths. [S2–S9]



Offline preparation feeds interactive prediction and analytical workflows.

Figure 1. Complete pipeline from trip and zone inputs to user-facing predictions and analytical answers. ML validation precedes artifact use; query validation precedes analytical execution.

Implementation boundary

The ETA artifact exists locally. Fare training code and saved notebook scores exist, but the fare artifact and historical lookup are absent from this checkout; the UI uses its rule baseline in their absence. The fleet LightGBM notebook is distinct from the dashboard’s implemented seven-day seasonal baseline. This diagram describes the full solution without implying that every experimental artifact is deployed.

Inference contracts

Fare returns a base-fare estimate in dollars, excluding a promise about final surcharges and tips. ETA returns predicted minutes and pickup time plus predicted duration. Demand returns hourly expected recorded pickups. The assistant returns a validated, bounded table with a chart or explanation; it does not train or alter the prediction models.

05 / Fare model methodology

Prediction problem and baselines

The target is `base_fare`, distinct from total settled charge. The experiment compares a global training median, an OD historical median, Ridge regression, LightGBM, and XGBoost. MAE in dollars is the primary selection measure because it expresses typical absolute pricing error in a directly interpretable unit. RMSE emphasizes larger errors; R-squared describes variance explained relative to a mean predictor. [S2]

Candidate	Configuration in source
Global / OD medians	Constant median or route median; global fallback for unseen routes
Ridge	alpha = 1.0; numeric input matrix
LightGBM	1,000 estimators; learning rate 0.05; 63 leaves; depth 8; seed 42
XGBoost	Absolute-error objective; 800 estimators; learning rate 0.05; depth 8; seed 42

Training workflow

The CLI loads up to 500,000 rows from each persisted split, enriches pre-trip features with route lookup statistics, trains the candidates, compares validation MAE, evaluates the selected estimator on the test subset, and serializes `UpfrontFarePipeline`. Historical routes with fewer than five training observations receive fallback statistics rather than unstable route medians.

Temporal and feature audit

The intended design is April–November training, December–January validation, and February–March testing. However, `make_splits.py` filters `pickup_month` using calendar-month numbers and Parquet row-group filtering. This does not establish the stated chronological boundaries: the feature examples show `pickup_month = 4` for April. The saved fare scores therefore remain experiment results, not a verified leakage-free out-of-time benchmark.

A defensible rerun must apply explicit `pickup_timestamp` predicates, assert non-overlap and date ordering, and record sample ranges and counts. The current loader takes leading rows rather than a representative sample across each full period. Lookup construction should also avoid self-target influence in training rows through out-of-fold or expanding-window encodings.

The interface asks for estimated distance, but historical `distance_miles` is recorded after the trip. Performance under a true pre-trip route-distance estimate has not been measured. Fare and ETA should therefore not share the same blanket “zero leakage” claim.

06 / Fare evaluation & interpretation

The following scores are transcribed from saved outputs in notebooks/03_fare_prediction.ipynb. Each saved raw subset contains 500,000 rows. The model was not retrained for this report. Interpret all scores with the temporal-split and distance-proxy qualifications on page 5. [S2]

Validation model	MAE (\$)	RMSE (\$)	R ²
Global median	12.1303	22.3159	-0.1875
OD median	3.8554	10.8388	0.7199
Ridge	6.0226	10.2138	0.7512
LightGBM	2.5913	7.3809	0.8701
XGBoost	2.8348	8.0500	0.8455

Saved selected-model test	MAE (\$)	RMSE (\$)	R ²
LightGBM	4.5969	7.0402	0.6793

What the comparison supports

LightGBM has the lowest stored validation MAE: approximately 32.8% lower than OD median and 8.6% lower than XGBoost. The OD baseline is already strong, indicating substantial route-level structure in fares. Ridge has lower RMSE than OD median but worse MAE, illustrating that candidate rankings depend on the operational error measure.

Test MAE is approximately 77.4% higher than validation MAE, while test RMSE is lower. This combination suggests a different error distribution or target mix; it should not be reduced to a claim of uniform performance degradation. Segment counts and residual quantiles are needed to establish the cause.

Error analysis and practical use

The notebook reports MAE of \$4.6104 for the non-airport flag and \$0.8994 for the airport flag. Without accompanying group sizes and a reviewed airport definition, these values should not imply universal airport reliability. Stratify by route frequency, tariff, borough, distance, and time of day before using model output as a customer commitment.

Metric definitions

MAE = $\text{mean}(|\text{actual} - \text{predicted}|)$. RMSE = square root of $\text{mean}((\text{actual} - \text{predicted})^2)$. $R^2 = 1 - \text{residual sum of squares} / \text{total sum of squares}$. Lower MAE/RMSE is better; higher R^2 is better. Negative R^2 is possible when a model performs worse than the corresponding mean baseline.

07 / Arrival-time estimation

The arrival estimator predicts duration in minutes and adds it to the requested pickup timestamp. Destination is assumed known before departure. The implementation uses LightGBM with an L1 objective, up to 1,200 trees, learning rate 0.04, 63 leaves, depth 10, seed 42, and validation early stopping after 75 rounds. Historical duration statistics are fitted on the training period. [S3]

The loader uses explicit timestamp cutoffs: training before 1 December 2025; validation from 1 December through 31 January; testing from 1 February 2026. It takes the first 500,000 chronologically sorted rows of each partition. The saved test evaluation retains 499,654 valid trips after duration filtering. These are period-prefix samples, not a full two-month test census.

Saved evaluation	MAE (min)	RMSE (min)	R²
Median baseline / validation	11.5427	18.7373	-0.1777
LightGBM / validation	7.2634	13.4872	0.3898
LightGBM / test	6.8857	12.7352	0.2842

Validation MAE improves about 37.1% over the median baseline. On the saved test sample, 31.11% of predictions fall within two minutes, 61.27% within five minutes, and 81.48% within ten minutes of actual duration.

Test segment	Rows	MAE (min)	Within 10 min
Rush hour	217,190	7.3958	80.26%
Non-rush hour	282,464	6.4935	82.42%
Short: 0–15 minutes	249,695	2.6759	98.79%
Medium: 15–45 minutes	222,430	8.1626	69.70%
Long: over 45 minutes	27,529	34.7525	19.71%

The long-trip failure mode is material: a single ETA should not be presented as a guaranteed arrival time. The duration segments above use actual outcomes for retrospective diagnosis; they are not pre-trip routing labels. Next development should target calibrated prediction intervals, long-trip coverage, and representative rolling-period evaluation.

The local artifact models/arrival_time_estimator.pkl is available. Reported performance comes from the saved notebook run; local artifact presence alone does not establish an exact artifact-to-metric version match.

08 / Fleet demand forecasting experiment

The Fleet Dispatcher notebook aggregates pickups into an hourly grid for ten high-volume zones over the intended 12-month window. It removes out-of-window timestamps and zero-fills missing zone-hours. Its 27 predictors combine zone identity, calendar encodings, lags of 1, 2, 3, 6, 12, 24, 48, 72 and 168 hours, and shifted rolling means over 3, 6, 12, 24 and 168 hours. [S4]

A LightGBM regressor uses up to 800 trees, learning rate 0.04, 63 leaves, L2 regularization 1.0, and seed 42. The split reserves the final 30 days for test and the preceding 14 days for validation. Evaluation uses the first 72 hours of those windows; predictions recursively replace unavailable future lag values.

Validation-selected strategies

Candidate strategies include LightGBM, last-observation persistence, a 168-hour seasonal baseline, and weighted blends. Validation selects a 75% LightGBM / 25% persistence blend for hours 1–24 and 25–48, and LightGBM for hours 49–72. Their saved validation WAPEs are 33.7%, 20.8%, and 13.4%, respectively.

Saved test horizon	MAE (pickups)	RMSE (pickups)	WAPE
1–24 hours	67.769	98.469	38.0%
25–48 hours	85.123	122.287	41.3%
49–72 hours	35.943	55.521	18.8%

The reported 32.7% mean test WAPE is the arithmetic mean of the three horizon WAPEs, not a pooled WAPE over all observations. $WAPE = \frac{\sum(|actual - predicted|)}{\sum(|actual|)}$; it is undefined when total actual demand is zero. Horizon-specific reporting is preferable to hiding variation in one number.

Operational output and evaluation limits

The notebook ranks predicted pickups over 6, 12, 24, and 72 hours. Its saved six-hour ranking places JFK Airport first, Times Square second, and LaGuardia third. These are historical forecast outputs, not current September demand observations.

Ten zones are selected using full-window volume, so a stricter prospective experiment should select them using training data only. One 72-hour origin does not establish seasonal robustness. The early previous-hour diagnostic uses observed preceding test hours, whereas the selected strategy uses recursive persistence; those baseline protocols should not be conflated. Fleet artifacts referenced by the notebook are absent locally, and this model is not wired into the dashboard demand page.

09 / Implemented dashboard demand baseline

The current Streamlit demand page uses a transparent seasonal-average baseline, separate from the fleet experiment. DuckDB reads pickup timestamp and origin ID from the cleaned Parquet file and aggregates hourly counts. Cached summaries support borough, zone, and two/four/eight-week history filters. Missing hours are zero-filled. [S5]

For each of the next 168 hours, the prediction is the mean recorded pickup count for matching weekdays and hours in the selected history. The final source calendar day is excluded because it may be incomplete. The chart reports daily totals, while the CSV preserves hourly forecasts; a heatmap and top-zone chart summarize the same selection.

Fresh all-zone check, 11 September 2026	Result
Historical window	4 February–31 March 2026 (56 days)
Recorded pickups	6,920,063
Mean daily pickups	123,572.55
Seven-day forecast total	865,007.88 pickups
Forecast dates	1–7 April 2026; anchored to dataset end
Held-out week	25–31 March 2026
Backtest training	4 February–24 March 2026
Held-out hourly MAE	466.34 aggregate pickups
Held-out aggregate WAPE	9.43%

Meaning of this result

This is a fresh fixed-origin check of the implemented function, with all 168 held-out hours excluded from the profile. It measures city-wide recorded pickup volume, not per-zone fleet prediction. Aggregation smooths local variation, so its 9.43% WAPE cannot be used to claim superiority over the fleet model’s 32.7% mean horizon WAPE.

The forecast is anchored to the historical data endpoint, not today. It includes no live weather, events, driver availability, or unmet requests. Zero pickup records are interpreted as zero observed demand, which would be inappropriate during an ingestion outage. Data completeness monitoring should precede operational use.

Verified application behavior

Streamlit page checks completed successfully for the assistant and demand modules after dependency repair. The demand page rendered four metrics and accepted a two-week history selection. These checks establish rendering and basic interaction, not comprehensive prediction accuracy or load-tested service capacity. [S10]

10 / Spatial hotspots & OD clustering

Spatial analysis builds per-zone behavioral descriptors from origin and destination activity, route averages, airport/cross-borough proportions, and five dayparts. StandardScaler normalizes selected inputs before K-Means. The pipeline also fits agglomerative clustering for comparison and exports zone profiles and origin-destination corridor summaries. Saved outputs contain 265 zone rows and 29,174 daypart corridor records. [S6]

Daypart	Hours
Morning rush	06:00–09:59
Midday	10:00–15:59
Evening rush	16:00–19:59
Late night	20:00–01:59
Overnight	02:00–05:59

Diagnostic	Saved value	Interpretation
Selected K-Means k	5	Operational taxonomy choice
Silhouette	0.2819	Higher is better; moderate separation
Davies–Bouldin	1.2735	Lower is better
Calinski–Harabasz	73.2096	Higher is better
Agglomerative silhouette, k=5	0.2399	Below K-Means at the same k
Best silhouette in k=2...8 search	0.3319 at k=3	Five is not the silhouette optimum

Interpretation and deployment role

The five labels describe commercial activity, nightlife, airport travel, residential/commuter behavior, and outer-borough low-density areas. These labels are assigned after clustering through feature-based rules, rather than learned from ground-truth zone classes. They are useful summaries for staging and service planning, not fixed identities for every trip in a zone.

The pipeline samples cleaned rows from monthly CSV chunks; therefore the corridor counts and cluster inputs should be treated as sample-derived activity unless explicitly reaggregated on the full dataset. Uniform chunk sampling does not guarantee proportional representation across months. The five-cluster choice trades stronger numerical separation at k=3 for finer operational categories.

Before automating dispatch by cluster, test month-to-month label stability, sparse-zone sensitivity, and performance of actual staging decisions. Use the profiles alongside real-time supply and travel constraints rather than as an optimizer.

11 / AI assistant & application delivery

Structured analytical workflow

The AI Mobility Assistant maps a question into a typed QueryIntent using the LLM client or local parsing logic. SchemaRegistry supplies valid fields and dataset-date semantics; entity resolution handles place-name ambiguity. IntentCompiler generates SQL, SQLGlott validates its structure, and DuckDB executes the approved query over the enriched Parquet view. Response and chart builders format the result, and an audit logger records interactions. [S7]

Layer	Responsibility
Pydantic intent schema	Validate structured entities, filters, aggregates and ordering
Schema / entity resolution	Resolve column names, zone aliases and date expressions
SQL compiler + validator	Restrict generated queries to supported analytical structure
DuckDB executor	Query the registered trip view and cap returned results at 1,000 rows
Response + chart + audit	Present results and retain traceable query information

The application’s default date interpretation is anchored to the dataset rather than the wall clock. Ambiguous requests can trigger clarification, and unsupported requests can be rejected. These controls reduce the risk of plausible but invalid queries; they do not establish that every natural-language request is interpreted correctly.

Evaluation status

The repository includes tests for ambiguity, intent routing, schema resolution, SQL validation, execution, and query planning, plus a golden-suite runner. No aggregate accuracy or complete test-pass rate was established during report preparation, so none is claimed. A production evaluation should score intent correctness, SQL validity, result correctness, clarification quality, rejection behavior, latency, and query cost separately.

Runtime and dependency management

The dashboard is served locally by Streamlit on port 8501. Recent missing-module failures were resolved by installing DuckDB, Pydantic, SQLGlott and associated dependencies in the same project virtual environment used by the server. Page checks passed and the server health endpoint returned “ok”; pip reported no broken requirements. This verifies the local runtime, not deployment resilience. [S10]

For repeatable delivery, pin a tested environment and package versioned model artifacts, feature definitions, data-window metadata and evaluation outputs together. Keep API credentials outside the report and source control. The training utilities also import fastparquet, which should be reconciled with the dependency manifest before rerunning that path.

12 / Business findings & scenario economics

The exported business tables are generated from an April 2025 raw-file analysis by default, with 3,782,338 trips in the fee decomposition. They are not full-year cleaned-data aggregates. Queens overnight occupied-trip revenue velocity is reported as \$127.81/hour versus \$82.83/hour for Manhattan midday. This reflects trip revenue relative to recorded trip time, not net driver earnings after waiting, empty travel and costs. [S8]

The Manhattan–Brooklyn table has 72,106 outbound trips and 28,909 returns: a difference of 43,197 and a 59.9% imbalance proxy. Without vehicle-linked trajectories, this is not an observed empty-return rate. Queens has more returns than outbound trips, producing a negative proxy; clip or relabel the metric before interpreting it as a rate.

Annual scenario component	Saved simulated value
Tip preset lift	\$16,679,850.00
Cash-to-digital recorded tip gain	\$4,549,050.00
Avoided empty-mile costs	\$4,741,764.30
Total modeled driver benefit	\$25,970,664.30
Average across 13,500 drivers	\$1,923.75 per driver
Empty miles avoided	7,295,022 miles

Assumptions behind the scenario

The calculation assumes 45.95 million annual trips, \$16.50 average base fare, 12% cash share, a 2.5-percentage-point tip lift, 25% cash conversion, 20% digital tipping, 18% outer-borough drop-offs, 70% empty returns, a 30% reduction in those returns, 4.2 avoided miles per affected trip, and \$0.65 cost per mile. These are scenario inputs, not jointly estimated causal effects.

Cash tips are not reliably recorded, so a recorded digital-tip increase is not automatically an increase in total driver income. The export’s \$17.25 million “platform EBITDA boost” uses an assumed processing-fee calculation without operating costs or an incremental counterfactual; it should not be represented as verified EBITDA.

Decision implications

Prioritize a measured dispatch-staging pilot with vehicle-level empty-mile outcomes and a clearly defined comparison group. Test payment-interface changes with voluntary, transparent choices and measure total earnings rather than recorded tips alone. Revenue decomposition also leaves about 5.35% outside its displayed categories; reconcile omitted fee components before financial reporting.

13 / Limitations, priorities & conclusions

Priority	Evidence / action required
1 · Reproducibility	Resolve existing merge-conflict markers in the EDA notebook; rerun explicit timestamp splits and save immutable data/model manifests.
2 · Fare validity	Use a true pre-trip distance estimate; validate disjoint date ranges and out-of-fold route statistics; regenerate the absent fare artifact.
3 · Demand validation	Use multiple rolling origins and training-only zone selection; report pooled and horizon WAPE with the same aggregation level.
4 · ETA reliability	Address long-trip errors; validate uncertainty intervals and geographically/time-stratified performance.
5 · Artifact integration	Package the fleet model and strategy metadata; distinguish its 72-hour zone forecast from the seven-day UI baseline.
6 · Operational measurement	Measure supply, waiting and empty mileage directly; validate scenario assumptions through pilots.

Scope of evidence

Saved notebook results document completed experiments on other runs, while fresh checks here establish the current cleaned-data count, the seasonal baseline backtest, and basic dashboard operation. No comprehensive training rerun, external-data verification, live dispatch experiment, or production load test was performed for this report. Artifact/version alignment remains to be recorded.

Conclusions

UrbanFlow demonstrates a coherent path from taxi trip data to passenger estimates and operational insight. Stored validation results favor LightGBM for fare prediction; the arrival model improves on a median baseline but has a pronounced long-trip weakness. The fleet notebook provides horizon-specific demand ranking, while the implemented seasonal baseline offers a transparent and testable dashboard forecast.

Spatial profiles and business tables identify useful hypotheses for staging and revenue improvement. Their value lies in narrowing operational decisions to test, rather than proving causal savings. The next milestone should be a reproducible, leakage-audited model release followed by a bounded dispatch pilot with measured service and driver outcomes.

The project is suitable as an integrated datathon prototype. Its technical contribution is the connection of audited data, interpretable baselines, tree-based prediction, spatial segmentation, structured query access and a working user interface—with the evaluation limits stated explicitly.

14 / Source register & reproduction notes

All sources below are project-local primary evidence. Paths are relative to the repository root. Bracketed source identifiers throughout the report map to this register; no external performance claims or third-party market assumptions were added.

ID	Source	Used for
S1	reports/data_quality_summary.json; anomaly_summary.csv; data_quality_decision_matrix.csv	Audit counts and task-specific treatments
S2	notebooks/03_fare_prediction.ipynb; src/models/fare_model.py; train_pipeline.py; src/features/fare_features.py; src/data/make_splits.py	Fare methods, saved metrics and split implementation
S3	notebooks/04_arrival_prediction.ipynb; src/models/arrival_model.py; train_arrival_pipeline.py	ETA features, saved metrics and segment diagnostics
S4	notebooks/04_3.1 The Fleet Dispatcher .ipynb	72-hour forecast, validation-selected strategies and saved test scores
S5	dashboard/demand.py	Implemented seven-day seasonal baseline and hourly aggregation
S6	src/analytics/flow_clustering.py; reports/clustering_metrics.json; zone_clusters.csv; daypart_od_flows.csv; notebooks/06_hotspot_od_analysis.ipynb	Clustering methods, taxonomy, diagnostics and outputs
S7	src/assistant/; config/mobility_schema.yaml; tests/assistant/	Structured query architecture and available test coverage
S8	src/analytics/business_analytics.py; reports/business_kpis.json; payment_tip_summary.csv; revenue_velocity.csv; deadhead_corridors.csv	Business data scope, assumptions and scenario calculations
S9	dashboard/app.py; requirements.txt; models/	UI integration, runtime dependencies and local artifact presence
S10	Fresh checks performed 11 September 2026; reports/technical_report_checks.json	Parquet count/date audit and aggregate-demand backtest; recorded runtime checks

Report build: .venv/Scripts/python.exe reports/build_technical_report.py
App: .venv/Scripts/python.exe -m streamlit run dashboard/app.py
Training: use the documented preparation and training modules after resolving the reproducibility issues on page 13. Reproduction commands are not evidence that a complete training rerun has already passed.

Prepared for Team DataCraft · Datathon 2026 · Report date: 11 September 2026